ONEDRAFT

CAD-AI — Aria Powered

Documentation & Drawing Generation Engine

Version 0.1

Status: Implementation Specification
Database: PostgreSQL
API: REST / JSON
API Version: `/api/v1`
Primary Model: Versioned OneDraft Project Model
AI Layer: Aria Powered
Documentation Standard: Configurable Architectural CAD Standard
Initial Sheet Standard: ARCH D
Primary Output: Coordinated Architectural Drawing Set
Document Authority: Versioned Project Model

---

1. ENGINE PURPOSE

The Documentation & Drawing Generation Engine converts the authoritative OneDraft computational project model into coordinated architectural documentation.

The engine does not treat the drawing as the project itself.

The authoritative relationship remains:

```
PROJECT
    ↓
PROJECT REQUIREMENTS
    ↓
COMPUTATIONAL PROJECT MODEL
    ↓
PARAMETRIC GEOMETRY
    ↓
COMPLIANCE MODEL
    ↓
VALIDATION
    ↓
DOCUMENTATION MODEL
    ↓
DRAWING SET
    ↓
DOCUMENT PACKAGE
```

Every generated drawing must remain traceable to the exact project state from which it was produced.

Each drawing generation operation therefore records:

```
Project_ID
Model_Version_ID
Revision_ID
Code_Manifest_ID
Validation_Run_ID
Generation_Run_ID
```

---

2. FUNDAMENTAL DOCUMENTATION PRINCIPLE

OneDraft shall not construct architectural drawings as disconnected graphic files.

The computational project model is authoritative.

The documentation engine creates controlled representations of that model.

Therefore:

```
ONE MODEL
    ↓
MANY VIEWS
    ↓
MANY DRAWINGS
    ↓
MANY SHEETS
    ↓
ONE COORDINATED DOCUMENT PACKAGE
```

A wall appearing on a floor plan, elevation, section, detail and schedule must remain the same authoritative model object.

A modification to that object must be capable of propagating to every affected representation.

---

3. DOCUMENTATION ENGINE ARCHITECTURE

The engine consists of:

```
Model Projection
View Generation
Drawing Generation
Annotation Engine
Dimension Engine
Schedule Engine
```

```
Sheet Composition Engine
Reference Coordination
Documentation Validation
Revision Propagation
Document Packaging
Document Provenance
```

The primary execution path is:

```
PROJECT MODEL
      ↓
MODEL VERSION
      ↓
DOCUMENTATION CONFIGURATION
      ↓
VIEW GENERATION
      ↓
GEOMETRIC PROJECTION
      ↓
GRAPHIC REPRESENTATION
      ↓
ANNOTATION
      ↓
DIMENSIONING
      ↓
SCHEDULE GENERATION
      ↓
SHEET COMPOSITION
      ↓
DOCUMENTATION VALIDATION
      ↓
PDF / DOCUMENT PACKAGE
```

---

4. DOCUMENTATION OBJECT MODEL

The documentation subsystem shall contain the following principal objects:

```
Views
Viewports
Drawings
Sheets
Sheet Templates
Dimensions
Annotations
Tags
Notes
Callouts
Section Markers
Elevation Markers
Detail References
Symbols
Legends
Graphics
Schedules
Drawing Dependencies
```

```
Generation Runs
Generation Results
```

All documentation objects shall reference the project and, where applicable, the model version and revision from which they were generated.

---

5. DOCUMENTATION DATABASE SCHEMA

The primary logical database schema is:

```
documentation
```

Existing tables:

```
documentation.views
documentation.drawings
documentation.sheets
documentation.sheet_drawings
documentation.sheet_templates
documentation.dimensions
documentation.annotations
documentation.schedules
```

Additional tables:

```
documentation.viewports
documentation.tags
documentation.notes
documentation.callouts
documentation.detail_references
documentation.section_markers
documentation.elevation_markers
documentation.symbol_instances
documentation.legends
documentation.graphics
documentation.drawing_dependencies
documentation.generation_runs
documentation.generation_results
```

---

6. DOCUMENTATION GENERATION RUN

Every documentation generation operation creates a generation record.

```
documentation.generation_runs
```

Fields:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
```

```
model_version_id UUID NOT NULL
code_manifest_id UUID
validation_run_id UUID
requested_by UUID
generation_type VARCHAR(64) NOT NULL
status VARCHAR(32) NOT NULL
configuration JSONB
started_at TIMESTAMPTZ
completed_at TIMESTAMPTZ
result_summary JSONB
created_at TIMESTAMPTZ NOT NULL
```

Generation types:

```
FULL_SET
PARTIAL_SET
PLAN_SET
ELEVATION_SET
SECTION_SET
DETAIL_SET
SCHEDULE_SET
SINGLE_DRAWING
SINGLE_SHEET
REDLINE_UPDATE
REVISION_UPDATE
DOCUMENT_PACKAGE
```

---

7. GENERATION STATUS

Generation operations use:

```
QUEUED
PREPARING
PROJECTING
ANNOTATING
DIMENSIONING
COMPOSING
VALIDATING
PACKAGING
COMPLETED
FAILED
CANCELLED
STALE
```

A generation becomes:

```
STALE
```

when its source model version is no longer the current project state.

Historical issued documentation remains preserved even when it becomes stale.

---

8. DOCUMENTATION CONFIGURATION

Each generation operation receives a documentation configuration.

The configuration may contain:

```
drawing_standard
sheet_standard
paper_size
scale_rules
line_weight_rules
text_rules
dimension_rules
annotation_rules
numbering_rules
title_block
revision_block
visibility_rules
schedule_rules
graphic_standard
```

The configuration must be versioned.

A historical drawing therefore retains the documentation configuration used to generate it.

---

9. VIEW MODEL

A view is a controlled projection of the computational project model.

Initial view types:

```
SITE_PLAN
FOUNDATION_PLAN
FLOOR_PLAN
ROOF_PLAN
REFLECTED_CEILING_PLAN
ELEVATION
BUILDING_SECTION
WALL_SECTION
DETAIL
3D_VIEW
DIAGRAM
SCHEDULE
```

Each view contains:

```
source_model_version
projection_definition
orientation
cut_plane
view_range
visibility_rules
scale
graphic_settings
```

```
annotation_rules
```

---

10. FLOOR PLAN GENERATION

A floor plan is generated from the corresponding level and its associated model elements.

The engine evaluates:

```
levels
spaces
walls
doors
windows
columns
stairs
ramps
vertical circulation
fixtures
equipment
furniture
structural elements
annotations
dimensions
grids
datums
```

The projection pipeline is:

```
LEVEL
 ↓
CUT PLANE
 ↓
CUT ELEMENTS
 ↓
VISIBLE ELEMENTS
 ↓
BEYOND ELEMENTS
 ↓
GRAPHIC HIERARCHY
 ↓
ANNOTATIONS
 ↓
DIMENSIONS
 ↓
FLOOR PLAN
```

---

11. PLAN CUT CONFIGURATION

A floor plan shall define:

```
cut_height
```

```
view_depth
top_view_limit
bottom_view_limit
projection_direction
```

The values are stored as part of the view configuration.

The engine shall not rely upon undocumented hard-coded cut planes.

---

12. FOUNDATION PLAN

Foundation documentation derives from the structural and architectural model.

The foundation plan may include:

```
footings
foundation walls
grade beams
piles
pile caps
columns
structural slabs
foundation elevations
drainage
waterproofing references
below-grade walls
```

Foundation documentation must reference the exact model version used for generation.

---

13. ROOF PLAN

Roof plans shall derive from:

```
roof elements
roof assemblies
parapets
slopes
drainage
mechanical penetrations
roof openings
skylights
equipment
edge conditions
```

The engine shall calculate and represent roof geometry from the parametric model rather than from independently drawn linework.

---

14. REFLECTED CEILING PLAN

Where required, reflected ceiling plans shall derive from:

```
spaces
ceiling assemblies
ceiling heights
lighting objects
mechanical diffusers
access panels
bulkheads
soffits
ceiling transitions
```

The reflected ceiling plan remains associated with the same project model.

---

15. ELEVATION GENERATION

Elevations are generated through controlled projection of the building model.

Initial orientations:

```
NORTH
SOUTH
EAST
WEST
```

Additional orientations may be defined as:

```
CUSTOM
INTERIOR
COURTYARD
PARTIAL
```

Each elevation records:

```
orientation
projection_plane
depth
visible_elements
level_datums
annotation_configuration
```

---

16. SECTION GENERATION

A section is generated from a section plane.

The section definition contains:

```
origin
direction
depth
cut_width
vertical_range
```

The engine performs:

```
SECTION PLANE
 ↓
GEOMETRIC INTERSECTION
 ↓
CUT ELEMENTS
 ↓
PROJECTED ELEMENTS
 ↓
DATUMS
 ↓
ANNOTATIONS
 ↓
DIMENSIONS
 ↓
SECTION DRAWING
```

---

17. SECTION DEPTH

The section depth determines which elements are visible beyond the cut plane.

The system shall distinguish:

```
CUT
PROJECTED
BEYOND
HIDDEN
```

according to the configured graphic standard.

---

18. DETAIL GENERATION

Details are enlarged controlled views of model geometry.

A detail definition contains:

```
source_view_id
source_objects
crop_region
detail_scale
detail_type
graphic_detail_level
annotation_rules
```

Initial detail types:

```
WALL_BASE
WALL_HEAD
WINDOW_HEAD
WINDOW_SILL
DOOR_JAMB
ROOF_EDGE
PARAPET
FOUNDATION
STAIR
GUARD
FLASHING
ASSEMBLY
```

---

19. MODEL DETAIL LEVEL

The geometry engine shall provide documentation-level representations.

A model element may therefore have:

```
SCHEMATIC
DESIGN
DOCUMENTATION
DETAIL
```

representations.

The documentation engine selects the appropriate representation according to the drawing type and scale.

---

20. VIEW DEPENDENCIES

Views may depend upon other views.

Example:

```
A101 FLOOR PLAN
      ↓
SECTION MARKER
      ↓
A301 BUILDING SECTION
      ↓
A501 WALL DETAIL
```

These relationships shall be persisted.

If the target view is deleted or invalidated, dependent references must be identified.

---

21. VIEWPORTS

A viewport defines how a view appears on a sheet.

```
documentation.viewports
```

Fields:

```
id UUID PRIMARY KEY
sheet_id UUID NOT NULL
view_id UUID NOT NULL
position JSONB NOT NULL
size JSONB
scale VARCHAR(64)
crop JSONB
display_settings JSONB
label_settings JSONB
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
```

---

22. DRAWING OBJECT

A drawing represents a documentation artifact derived from a view.

Each drawing contains:

```
drawing_number
title
discipline
scale
view_id
model_version_id
revision_id
status
```

A drawing is therefore a generated representation, not an independent model.

---

23. SHEET OBJECT

A sheet represents the composed documentation page.

A sheet may contain:

```
drawings
viewports
notes
legends
schedules
symbols
revision information
```

```
title block
```

The sheet is the principal printable page object.

---

24. SHEET SIZE SUPPORT

The initial engine shall support:

```
ARCH_A
ARCH_B
ARCH_C
ARCH_D
ARCH_E
ANSI_A
ANSI_B
ANSI_C
ANSI_D
ANSI_E
```

Templates determine:

```
page dimensions
orientation
margins
printable area
title block
revision block
graphic standard
```

---

25. ARCH D

ARCH D is a first-class supported sheet format.

Standard dimensions:

```
24 × 36 inches
```

The template may support:

```
LANDSCAPE
PORTRAIT
```

depending upon project configuration.

---

26. TITLE BLOCK

The title block shall derive authoritative information from the project model.

Initial fields:

```
project_name
project_address
client
project_number
drawing_number
drawing_title
discipline
scale
date
revision
sheet_number
drawn_by
checked_by
approved_by
```

Duplicate manual entry of authoritative project data should be avoided.

---

27. REVISION BLOCK

Every issued revision shall be capable of displaying:

```
revision_number
revision_date
description
author
checker
approval
```

The revision block must correspond to the authoritative revision record.

---

28. GRAPHIC HIERARCHY

The documentation engine shall use semantic graphic classes.

Initial classes:

```
CUT_PRIMARY
CUT_SECONDARY
VISIBLE_PRIMARY
VISIBLE_SECONDARY
OVERHEAD
BEYOND
HIDDEN
REFERENCE
```

```
DIMENSION
ANNOTATION
HATCH
SYMBOL
```

Graphic rules are configurable.

---

29. LINE WEIGHTS

Line weights are associated with semantic roles.

The system shall not depend on arbitrary element-specific lineweight values.

Example:

```
CUT_PRIMARY → strongest
CUT_SECONDARY → strong
VISIBLE_PRIMARY → medium
VISIBLE_SECONDARY → light
OVERHEAD → lighter
ANNOTATION → controlled
DIMENSION → controlled
```

Exact values belong to the active graphic standard.

---

30. MATERIAL HATCHING

Material representations shall use parametric hatch definitions.

Initial material categories:

```
CONCRETE
EARTH
INSULATION
MASONRY
WOOD
STEEL
GYPSUM
ROOFING
GRAVEL
```

The hatch should derive from the material or assembly definition wherever possible.

---

31. DIMENSION ENGINE

Dimensions shall be generated from authoritative geometry.

Supported types:

```
LINEAR
ALIGNED
ANGULAR
RADIAL
DIAMETER
ELEVATION
BASELINE
CONTINUOUS
ORDINATE
```

Each dimension retains references to the geometry used to calculate its displayed value.

---

32. ASSOCIATIVE DIMENSIONS

Associative dimensions shall contain:

```
source_objects
reference_points
geometry_version
calculated_value
unit
tolerance
```

When geometry changes, the dimension shall be:

```
RECALCULATED
```

or:

```
INVALIDATED
```

It must never silently retain an obsolete measurement.

---

33. ANNOTATION ENGINE

The annotation engine supports:

```
GENERAL_NOTES
KEYNOTES
LABELS
ROOM_TAGS
DOOR_TAGS
WINDOW_TAGS
MATERIAL_TAGS
LEVEL_MARKERS
GRID_LABELS
SECTION_MARKERS
ELEVATION_MARKERS
```

```
DETAIL_MARKERS
```

Annotations shall be associated with model objects whenever practical.

---

34. ROOM TAGGING

Room tags derive from the authoritative space object.

The tag may display:

```
space_number
name
area
occupancy
```

The area must be calculated from current authoritative geometry.

A manually typed area shall not override the model-derived value without an explicit exception mechanism.

---

35. DOOR TAGGING

Door tags derive from the authoritative door element.

A door identity must remain consistent between:

```
floor plan
door schedule
elevation
section
detail
```

Example:

```
D-101
```

must resolve to one model door object.

---

36. WINDOW TAGGING

Window tags follow the same model identity principle.

A window identity must remain coordinated between:

```
plans
elevations
```

```
sections
schedules
details
```

---

37. LEVEL DATUMS

Level markers derive from:

```
level_code
name
elevation
sequence
```

All applicable drawings shall receive coordinated datum information.

---

38. GRID SYSTEM

Architectural and structural grids shall derive from the model grid system.

Grid labels must remain coordinated across:

```
plans
sections
elevations
details
```

---

39. SCHEDULE ENGINE

Schedules are generated from model queries.

Initial schedules:

```
DOOR SCHEDULE
WINDOW SCHEDULE
ROOM SCHEDULE
FINISH SCHEDULE
EQUIPMENT SCHEDULE
FIXTURE SCHEDULE
MATERIAL SCHEDULE
ASSEMBLY SCHEDULE
```

Future disciplines may introduce:

```
STRUCTURAL
MECHANICAL
ELECTRICAL
PLUMBING
```

```
LIFE SAFETY
ACCESSIBILITY
ENERGY
```

---

40. SCHEDULE ROW PROVENANCE

Every schedule row shall retain its source model identity.

The relationship is:

```
MODEL OBJECT
 ↓
SCHEDULE QUERY
 ↓
SCHEDULE ROW
 ↓
DRAWING
```

This prevents schedule information from becoming detached from the computational model.

---

41. AUTOMATIC COORDINATION

The documentation engine shall detect inconsistencies between representations.

Examples:

```
DOOR_ON_PLAN_NOT_IN_SCHEDULE
WINDOW_ON_ELEVATION_NOT_IN_SCHEDULE
ROOM_TAG_WITHOUT_SPACE
BROKEN_SECTION_REFERENCE
BROKEN_DETAIL_REFERENCE
DIMENSION_WITHOUT_GEOMETRY
STALE_VIEW
STALE_DRAWING
STALE_SCHEDULE
DUPLICATE_DRAWING_REFERENCE
```

Each condition becomes a documentation validation result.

---

42. DOCUMENTATION VALIDATION

The engine shall execute:

```
MODEL_COORDINATION
VIEW_COORDINATION
ANNOTATION_VALIDATION
DIMENSION_VALIDATION
```

```
SCHEDULE_VALIDATION
REFERENCE_VALIDATION
SHEET_VALIDATION
REVISION_VALIDATION
OUTPUT_VALIDATION
```

---

43. DOCUMENTATION VALIDATION SEVERITY

Results shall use:

```
ERROR
WARNING
INFO
```

An `ERROR` prevents final issuance unless explicitly resolved or formally overridden under the applicable workflow.

A `WARNING` may permit generation but remains recorded.

---

44. DRAWING NUMBERING

Drawing numbering shall be template-driven.

Initial architectural examples:

```
A001
A002
A101
A102
A201
A202
A301
A302
A401
A501
```

The numbering engine must not hard-code these values.

---

45. DISCIPLINE SYSTEM

Initial disciplines:

```
A
S
```

```
M
E
P
FP
L
C
G
```

Architecture is the first implementation priority.

The underlying documentation architecture shall support multidisciplinary coordination.

---

46. DRAWING SET ORGANIZATION

A configurable architectural drawing set may contain:

```
GENERAL
SITE
PLANS
ELEVATIONS
SECTIONS
DETAILS
SCHEDULES
LEGENDS
```

A default numbering strategy may use:

```
A000–A099 GENERAL
A100–A199 SITE / PLANS
A200–A299 ELEVATIONS
A300–A399 SECTIONS
A400–A499 DETAILS
A500–A599 SCHEDULES / LEGENDS
```

Templates remain authoritative for actual project numbering.

---

47. SHEET COMPOSITION ENGINE

The sheet composer receives:

```
sheet_template
drawings
views
viewports
annotations
schedules
notes
legends
symbols
revision_data
```

and produces:

```
COMPOSED_SHEET
```

The composer must evaluate:

```
sheet boundaries
viewport placement
scale
annotation spacing
title block
revision block
schedule overflow
graphic hierarchy
```

---

48. AUTOMATIC LAYOUT

Aria may optimize sheet composition subject to:

```
sheet boundaries
viewport requirements
scale requirements
drawing hierarchy
readability
minimum spacing
title block requirements
project standards
```

Aria shall not alter authoritative model geometry merely to solve a documentation layout problem.

---

49. COLLISION DETECTION

The documentation engine shall detect:

```
TEXT/TEXT
TEXT/DIMENSION
TEXT/GEOMETRY
TAG/TAG
DIMENSION/DIMENSION
VIEWPORT/VIEWPORT
SCHEDULE/BOUNDARY
TITLE_BLOCK/CONTENT
```

The system may automatically reposition documentation objects when doing so preserves their semantic relationships.

Unresolvable collisions become validation results.

---

50. SCALE CONTROL

Initial supported scales include:

```
1:50
1:100
1:200
1:500
1/4" = 1'-0"
1/8" = 1'-0"
1/16" = 1'-0"
```

Scale shall be represented as structured configuration.

---

51. SCALE VALIDATION

The engine verifies consistency between:

```
view_scale
viewport_scale
drawing_scale
sheet_label
```

An inconsistency becomes:

```
INVALID_SCALE
```

and is recorded in documentation validation.

---

52. DETAIL CALLOUTS

A detail callout contains:

```
source_view_id
target_detail_view_id
reference_number
position
```

The target detail must exist in the documentation model.

Deleting or invalidating the target creates a dependency violation.

---

53. SECTION MARKERS

A section marker contains:

```
source_view_id
target_section_view_id
section_plane
reference_number
position
```

The marker and section must remain synchronized.

---

54. ELEVATION MARKERS

An elevation marker contains:

```
source_view_id
target_elevation_view_id
orientation
reference_number
position
```

Orphaned elevation markers shall be detected automatically.

---

55. LEGENDS

Legends may contain:

```
symbols
line_types
material_graphics
abbreviations
keynotes
```

Legends may be:

```
PROJECT_DEFINED
TEMPLATE_DEFINED
STANDARD_DEFINED
```

---

56. NOTES ENGINE

The notes engine supports:

```
PROJECT_NOTES
GENERAL_NOTES
KEYED_NOTES
VIEW_NOTES
DETAIL_NOTES
SHEET_NOTES
```

Notes may originate from:

```
USER
TEMPLATE
MODEL
ARIA
```

The provenance source shall be retained.

---

57. ARIA DOCUMENTATION CAPABILITIES

Aria may execute documentation operations through controlled application capabilities.

Supported operations include:

```
CREATE_VIEW
MODIFY_VIEW
CREATE_DRAWING
CREATE_SHEET
PLACE_VIEWPORT
GENERATE_DIMENSIONS
GENERATE_ANNOTATIONS
GENERATE_SCHEDULE
CREATE_DETAIL
CREATE_SECTION
CREATE_ELEVATION
ORGANIZE_SHEETS
DETECT_COORDINATION_ERRORS
RESOLVE_LAYOUT_CONFLICTS
REGENERATE_DOCUMENTATION
```

---

58. ARIA DOCUMENTATION BOUNDARY

Aria shall not directly:

```
write SQL
modify PostgreSQL directly
replace production files directly
modify production permissions
modify billing records
modify infrastructure configuration
```

Aria operates through OneDraft domain services and documented capabilities.

---

59. ARIA DOCUMENTATION REQUEST

Endpoint:

```
POST /api/v1/projects/{project_id}/aria/documentation
```

Request:

```
{
  "revision_id": "uuid",
  "instruction": "Generate the architectural drawing set for the current model."
}
```

Response:

```
{
  "data": {
    "generation_run_id": "uuid",
    "status": "QUEUED"
  }
}
```

---

60. DRAWING GENERATION REQUEST

Endpoint:

```
POST /api/v1/projects/{project_id}/drawings/generate
```

Request:

```
{
  "revision_id": "uuid",
  "drawing_types": [
    "PLAN",
    "ELEVATION",
    "SECTION",
    "DETAIL",
    "SCHEDULE"
  ],
  "sheet_template": "ARCH_D"
}
```

Response:

```
{
  "data": {
    "job_id": "uuid",
    "generation_run_id": "uuid",
    "status": "QUEUED"
  }
}
```

---

61. SINGLE DRAWING REGENERATION

Endpoint:

```
POST /api/v1/projects/{project_id}/drawings/{drawing_id}/regenerate
```

Request:

```
{
  "revision_id": "uuid",
  "model_version_id": "uuid"
}
```

The service verifies that the referenced project state remains valid before regeneration.

---

62. SHEET GENERATION

Endpoint:

```
POST /api/v1/projects/{project_id}/sheets/generate
```

Request:

```
{
  "revision_id": "uuid",
  "sheet_template": "ARCH_D",
  "drawing_ids": [
    "uuid"
  ]
}
```

Response:

```
{
  "data": {
    "job_id": "uuid",
    "generation_run_id": "uuid",
    "status": "QUEUED"
  }
}
```

---

63. DOCUMENT PACKAGE GENERATION

Endpoint:

```
POST /api/v1/projects/{project_id}/documents/generate
```

Request:

```
{
  "revision_id": "uuid",
  "format": "PDF",
  "paper_size": "ARCH_D",
  "include": [
    "PLANS",
    "ELEVATIONS",
    "SECTIONS",
    "DETAILS",
    "SCHEDULES",
    "NOTES"
  ]
}
```

---

64. OUTPUT FORMATS

Initial authoritative output:

```
PDF
```

Future output adapters may support:

```
SVG
DXF
DWG
IFC
PNG
JPG
```

Subject to implementation and licensing requirements.

---

65. VECTOR-FIRST DOCUMENTATION

Where technically feasible, OneDraft shall generate vector documentation rather than raster-only drawing output.

Vector output preserves:

```
line precision
text quality
scalability
print quality
machine readability
```

Rasterization may be used for compatible embedded content where required.

---

66. PDF GENERATION PIPELINE

The PDF pipeline is:

```
PROJECT MODEL
 ↓
MODEL VERSION
 ↓
VIEW
 ↓
DRAWING
 ↓
SHEET
 ↓
VECTOR GRAPHICS
 ↓
PDF
```

The resulting PDF must preserve:

```
sheet number
drawing title
project identification
revision
generation state
document metadata
```

---

67. PARTIAL REGENERATION

The engine shall support selective documentation regeneration.

Example:

```
CHANGE WINDOW W-204
```

Dependency analysis may identify:

```
A102
A201
A301
A501
WINDOW SCHEDULE
```

as affected while leaving unrelated documentation unchanged.

---

68. FULL REGENERATION

A complete drawing set may be regenerated through:

```
POST /api/v1/projects/{project_id}/drawings/generate
```

with:

```
generation_type = FULL_SET
```

Full regeneration creates a new documentation generation run associated with the selected model version.

---

69. DOCUMENTATION DEPENDENCY GRAPH

The documentation system shall maintain a dependency graph.

Example:

```
MODEL ELEMENT
      ↓
VIEW
      ↓
DRAWING
      ↓
SHEET
      ↓
DOCUMENT PACKAGE
```

Additional relationships may include:

```
VIEW
 ↓
SECTION MARKER
 ↓
SECTION VIEW
```

and:

```
SPACE
 ↓
ROOM TAG
 ↓
ROOM SCHEDULE
```

---

70. REVISION PROPAGATION

When a model object changes:

```
MODEL CHANGE
 ↓
```

```
AFFECTED OBJECTS
  ↓
AFFECTED VIEWS
  ↓
AFFECTED DRAWINGS
  ↓
AFFECTED SHEETS
  ↓
AFFECTED SCHEDULES
  ↓
DOCUMENTATION INVALIDATION
```

The engine determines the minimum affected documentation set.

---

71. DOCUMENTATION INVALIDATION

A documentation object becomes invalid when:

```
source geometry changes
source model version changes
referenced object is deleted
required view changes
required code state changes
required annotation becomes unresolved
```

The system must explicitly mark invalid documentation rather than silently presenting stale content as current.

---

72. MODEL VERSION CHECK

Every documentation generation job records:

```
project_id
model_version_id
revision_id
```

If the model changes before completion, the worker verifies that its source state remains valid.

If not:

```
generation_status = STALE
```

The worker must not publish stale output as current documentation.

---

73. REVISION CHECK

Every drawing generation operation verifies:

```
expected_revision
current_revision
```

If they differ, the system shall require regeneration against the current revision or explicitly identify the historical revision being requested.

---

74. CODE MANIFEST RELATIONSHIP

Where documentation is intended for compliance or issuance, the drawing set shall record:

```
code_manifest_id
```

This establishes which regulatory source state accompanied the documentation.

---

75. VALIDATION RELATIONSHIP

A final document package must identify the applicable validation state:

```
validation_run_id
```

The validation run must correspond to the same:

```
project
revision
model_version
code_manifest
```

as the documentation package.

---

76. DOCUMENT PROVENANCE

The complete provenance chain is:

```
PROJECT
 ↓
MODEL VERSION
 ↓
REVISION
 ↓
CODE MANIFEST
 ↓
```

```
VALIDATION RUN
 ↓
DOCUMENTATION GENERATION RUN
 ↓
VIEWS
 ↓
DRAWINGS
 ↓
SHEETS
 ↓
DOCUMENT PACKAGE
 ↓
HASH
```

This permits reconstruction of how an issued document was produced.

---

77. DOCUMENT HASHING

Every finalized document package shall receive a cryptographic hash.

Initial implementation:

```
SHA-256
```

Conceptually:

```
document_hash = SHA256(package)
```

The hash is stored against:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
generation_run_id
```

---

78. HISTORICAL DOCUMENTATION

Issued historical documentation shall be immutable.

For example:

```
REVISION 1
```

remains preserved when:

```
REVISION 2
```

is generated.

The system shall never silently replace an issued historical document.

---

79. DOCUMENT PACKAGE STRUCTURE

A generated architectural package may contain:

```
00_COVER
01_GENERAL
02_SITE
03_PLANS
04_ELEVATIONS
05_SECTIONS
06_DETAILS
07_SCHEDULES
08_NOTES
09_APPENDICES
```

The exact structure is configurable.

---

80. PACKAGE MANIFEST

Every generated package shall contain a machine-readable manifest.

Conceptually:

```
manifest.json
```

The manifest records:

```
project_id
revision_id
model_version_id
code_manifest_id
validation_run_id
generation_run_id
document_hash
sheet_list
drawing_list
generation_timestamp
software_version
documentation_standard
```

---

81. DOCUMENT PACKAGE INTEGRITY

Before finalization, the engine verifies:

```
all required sheets exist
```

```
all required drawings exist
all references resolve
all sheets are valid
all required schedules exist
all required annotations resolve
all document metadata is present
all source versions are consistent
```

Only after successful integrity validation may the package enter:

```
READY_FOR_REVIEW
```

---

82. CUSTOMER REVIEW STATE

The documentation workflow shall support:

```
DRAFT
INTERNAL_REVIEW
CUSTOMER_REVIEW
REDLINE
REVISION
VALIDATION
READY_FOR_ACCEPTANCE
ACCEPTED
ISSUED
SUPERSEDED
```

The exact transition rules are controlled by the revision and acceptance services.

---

83. REDLINE INTEGRATION

A customer redline shall reference:

```
project_id
revision_id
drawing_id
target_object_id
markup_geometry
instruction
```

The redline enters:

```
OPEN
```

The engine does not consider the redline resolved merely because the markup has been removed.

Resolution requires:

```
MODEL CHANGE
  ↓
```

```
DOCUMENTATION REGENERATION
 ↓
VALIDATION
 ↓
REDLINE RESOLUTION
```

---

84. REDLINE RESOLUTION

A resolved redline records:

```
redline_id
command_id
implemented_changes
validation_results
resolved_by
resolved_at
```

This provides traceability between customer feedback and actual model modification.

---

85. DOCUMENTATION REGRESSION TESTING

Every significant documentation-engine change shall be tested against known projects.

The regression system should compare:

```
model state
view state
drawing state
sheet state
document package
```

against expected outputs.

The comparison may use:

```
geometry comparison
metadata comparison
reference comparison
visual comparison
hash comparison
```

---

86. VISUAL REGRESSION

The engine shall support visual regression testing for generated sheets.

A test may compare:

```
expected A101
```

against:

```
generated A101
```

and identify:

```
geometry changes
missing elements
annotation changes
layout changes
scale changes
title block changes
```

Visual comparison is supplemental to semantic validation and does not replace model-level validation.

---

87. DOCUMENTATION ENGINE WORKER

Expensive generation operations shall execute asynchronously.

Conceptually:

```
API
 ↓
JOB QUEUE
 ↓
DOCUMENTATION WORKER
 ↓
MODEL SERVICE
 ↓
GEOMETRY SERVICE
 ↓
DOCUMENTATION SERVICE
 ↓
PDF ENGINE
 ↓
OBJECT STORAGE
```

The worker records progress against the generation run.

---

88. DOCUMENTATION JOB PROGRESS

Progress stages may include:

```
0% PREPARING
10% LOADING MODEL
25% GENERATING VIEWS
```

```
40% GENERATING DRAWINGS
55% GENERATING ANNOTATIONS
65% GENERATING SCHEDULES
75% COMPOSING SHEETS
85% VALIDATING
95% PACKAGING
100% COMPLETED
```

Progress values are informational and must not be interpreted as a guarantee of remaining execution time.

---

89. OBJECT STORAGE

Generated files shall not be stored directly inside PostgreSQL.

PostgreSQL stores:

```
object_reference
file_hash
file_size
mime_type
generation_run_id
```

The actual document artifact is stored through the OneDraft object-storage subsystem.

---

90. DOCUMENT ARTIFACT RECORD

Generated artifacts should contain:

```
id UUID PRIMARY KEY
project_id UUID NOT NULL
revision_id UUID NOT NULL
model_version_id UUID NOT NULL
generation_run_id UUID NOT NULL
artifact_type VARCHAR(64) NOT NULL
storage_reference TEXT NOT NULL
mime_type VARCHAR(128)
file_size BIGINT
content_hash VARCHAR(128)
created_at TIMESTAMPTZ NOT NULL
```

---

91. API DOCUMENTATION RESOURCES

The API shall expose:

```
Views
```

```
Viewports
Drawings
Sheets
Schedules
Annotations
Dimensions
Redlines
Generation Runs
Document Packages
Jobs
```

under:

```
/api/v1
```

---

92. VIEW ENDPOINTS

Initial endpoints:

```
POST   /api/v1/projects/{project_id}/views
GET    /api/v1/projects/{project_id}/views
GET    /api/v1/projects/{project_id}/views/{view_id}
PATCH  /api/v1/projects/{project_id}/views/{view_id}
DELETE /api/v1/projects/{project_id}/views/{view_id}
```

---

93. DRAWING ENDPOINTS

Initial endpoints:

```
POST   /api/v1/projects/{project_id}/drawings/generate
GET    /api/v1/projects/{project_id}/drawings
GET    /api/v1/projects/{project_id}/drawings/{drawing_id}
POST   /api/v1/projects/{project_id}/drawings/{drawing_id}/regenerate
```

---

94. SHEET ENDPOINTS

Initial endpoints:

```
POST   /api/v1/projects/{project_id}/sheets
GET    /api/v1/projects/{project_id}/sheets
GET    /api/v1/projects/{project_id}/sheets/{sheet_id}
PATCH  /api/v1/projects/{project_id}/sheets/{sheet_id}
POST   /api/v1/projects/{project_id}/sheets/generate
```

---

95. DOCUMENT ENDPOINTS

Initial endpoints:

```
POST /api/v1/projects/{project_id}/documents/generate
GET  /api/v1/projects/{project_id}/documents
GET  /api/v1/projects/{project_id}/documents/{document_id}
```

---

96. GENERATION RUN ENDPOINTS

```
GET /api/v1/projects/{project_id}/generation-runs
GET /api/v1/projects/{project_id}/generation-runs/{generation_run_id}
```

---

97. DOCUMENTATION PERMISSIONS

Initial scopes:

```
drawing:read
drawing:generate

sheet:read
sheet:write
sheet:generate

documentation:read
documentation:write
documentation:generate

document:read
document:generate
```

The authorization service must evaluate both:

```
scope
```

and:

```
organization/project membership
```

---

98. TRANSACTION BOUNDARY

Documentation state changes must occur through controlled domain transactions.

Conceptually:

```
BEGIN
 ↓
VERIFY PROJECT
 ↓
VERIFY REVISION
 ↓
VERIFY MODEL VERSION
 ↓
VERIFY AUTHORIZATION
 ↓
CREATE GENERATION RECORD
 ↓
COMMIT
```

Long-running generation then occurs asynchronously.

The generated result is finalized through a controlled completion transaction.

---

99. GENERATION FINALIZATION

Finalization verifies:

```
source model unchanged
revision unchanged
required views generated
required drawings generated
required sheets generated
validation completed
document integrity verified
```

The system then records:

```
generation_status = COMPLETED
```

and makes the package available for the next workflow stage.

---

100. STALE OUTPUT PROTECTION

A generated artifact shall never automatically become the current project documentation merely because generation completed.

The system verifies:

```
generated_model_version == requested_model_version
generated_revision == requested_revision
```

If not:

```
STALE
```

is recorded.

---

101. DRAWING COORDINATION TEST

The first documentation integration test shall verify:

```
CREATE PROJECT
 ↓
CREATE BUILDING
 ↓
CREATE LEVELS
 ↓
CREATE SPACES
 ↓
CREATE WALLS
 ↓
CREATE DOORS
 ↓
CREATE WINDOWS
 ↓
CREATE ROOF
 ↓
CREATE FOUNDATION
 ↓
GENERATE FLOOR PLAN
 ↓
GENERATE ELEVATIONS
 ↓
GENERATE SECTION
 ↓
GENERATE DETAILS
 ↓
GENERATE SCHEDULES
 ↓
COMPOSE ARCH D SHEETS
 ↓
VALIDATE DOCUMENTATION
 ↓
GENERATE PDF
```

---

102. DOCUMENTATION CHANGE TEST

The next test shall modify a model element.

Example:

```
MOVE WALL W-101
```

The system must identify:

```
affected views
```

```
affected dimensions
affected annotations
affected schedules
affected drawings
affected sheets
```

Only affected documentation must be invalidated or regenerated where dependency analysis permits.

---

103. REDLINE DOCUMENTATION TEST

The documentation regression sequence shall include:

```
GENERATE DRAWING
 ↓
CREATE REDLINE
 ↓
EXECUTE ARIA COMMAND
 ↓
CREATE NEW REVISION
 ↓
REGENERATE AFFECTED DOCUMENTATION
 ↓
VALIDATE
 ↓
RESOLVE REDLINE
 ↓
GENERATE NEW PACKAGE
```

The previous drawing set must remain preserved.

---

104. FINAL PACKAGE TEST

The package test shall verify that:

```
PDF EXISTS
 ↓
ALL REQUIRED SHEETS EXIST
 ↓
DRAWING NUMBERS ARE UNIQUE
 ↓
REFERENCES RESOLVE
 ↓
SCHEDULES MATCH MODEL
 ↓
ANNOTATIONS RESOLVE
 ↓
REVISION DATA MATCHES
 ↓
MODEL VERSION MATCHES
 ↓
CODE MANIFEST MATCHES
```

```
  ↓
VALIDATION RUN MATCHES
  ↓
HASH IS RECORDED
```

---

105. DOCUMENTATION ENGINE SUCCESS CRITERIA

The Documentation & Drawing Generation Engine is successful when it can:

```
Create views

Generate floor plans

Generate foundation plans

Generate roof plans

Generate elevations

Generate sections

Generate details

Generate schedules

Generate dimensions

Generate annotations

Generate room tags

Generate door tags

Generate window tags

Generate section references

Generate elevation references

Compose ARCH D sheets

Maintain drawing numbering

Maintain title blocks

Maintain revision blocks

Detect documentation conflicts

Propagate model revisions

Regenerate affected documentation
```

```
Generate complete PDF packages

Preserve historical documentation

Record provenance

Hash final packages
```

---

106. ARCHITECTURAL DOCUMENTATION SUCCESS TEST

The minimum complete workflow is:

```
PROJECT
  ↓
MODEL
  ↓
GEOMETRY
  ↓
COMPLIANCE
  ↓
VALIDATION
  ↓
VIEWS
  ↓
DRAWINGS
  ↓
SHEETS
  ↓
SCHEDULES
  ↓
DOCUMENTATION VALIDATION
  ↓
PDF
  ↓
CUSTOMER REVIEW
  ↓
REDLINE
  ↓
REVISION
  ↓
REGENERATION
  ↓
FINAL PACKAGE
```

This workflow must operate without requiring the customer or frontend to manually duplicate the computational project model.

---

107. DOCUMENTATION SYSTEM OF RECORD

The documentation system of record is:

```
MODEL
 ↓
VIEW DEFINITION
 ↓
DRAWING DEFINITION
 ↓
SHEET DEFINITION
 ↓
DOCUMENT PACKAGE
```

The rendered PDF is a derived artifact.

The model remains authoritative.

---

108. DOCUMENTATION RECONSTRUCTION TEST

Given:

```
Project_ID
Revision_ID
Model_Version_ID
Code_Manifest_ID
Generation_Run_ID
```

the system must be capable of reconstructing:

```
views
drawings
sheets
schedules
annotations
dimensions
references
document package
```

If the documentation cannot be reconstructed from its recorded source state, the documentation provenance architecture is incomplete.

---

109. IMPLEMENTATION ARTIFACTS

The next implementation package shall consist of:

Artifact A

`documentation_schema.sql`

Database tables and indexes for the documentation engine.

Artifact B

`documentation_models.py`

Python/Pydantic documentation-domain models.

Artifact C

`documentation_service.py`

Core documentation generation and dependency service.

Artifact D

`drawing_generator.py`

View projection and architectural drawing generation engine.

Artifact E

`sheet_composer.py`

Sheet layout, viewport placement, annotation composition and title-block generation.

Artifact F

`schedule_generator.py`

Model-driven schedule generation.

Artifact G

`documentation_validator.py`

Documentation coordination and integrity validation.

Artifact H

`document_package_service.py`

Final package assembly, hashing and object-storage registration.

---

110. IMPLEMENTATION ORDER

The implementation sequence shall be:

1. Documentation schema
2. Documentation domain types
3. View service
4. Projection service
5. Drawing generator
6. Annotation engine
7. Dimension engine
8. Schedule generator
9. Sheet composer
10. Reference coordination
11. Documentation validator
12. PDF renderer
13. Document package service
14. Provenance and hashing
15. Redline integration
16. Regression tests
17. End-to-end documentation test

---

111. INTEGRATION WITH EXISTING ONEDRAFT STACK

The Documentation Engine sits above the previously established systems:

```
IDENTITY
  ↓
PROJECT
  ↓
DOMAIN MODEL
  ↓
PARAMETRIC GEOMETRY
```

```
  ↓
COMPLIANCE
  ↓
VALIDATION
  ↓
DOCUMENTATION
```

It therefore consumes authoritative outputs from the preceding layers and does not create competing representations of those systems.

---

112. ARCHITECTURAL DRAWING PRINCIPLE

OneDraft shall generate architectural drawings as computational documentation.

The drawing therefore represents:

```
WHAT THE MODEL IS
```

at a particular:

```
REVISION
```

under a particular:

```
CODE MANIFEST
```

and through a particular:

```
DOCUMENTATION STANDARD
```

at a particular:

```
MODEL VERSION
```

---

113. FINAL ARCHITECTURAL DECISION

OneDraft documentation shall not be implemented as:

```
IMAGE GENERATION
+
PDF EXPORT
+
STATIC CAD FILES
```

It shall be implemented as:

```
PARAMETRIC PROJECT MODEL
```

```
+
GEOMETRIC PROJECTION
+
DOCUMENTATION MODEL
+
DRAWING GENERATION
+
SHEET COMPOSITION
+
COORDINATION
+
VALIDATION
+
REVISION CONTROL
+
DOCUMENT PROVENANCE
```

The architectural drawing is therefore a controlled derivative of the OneDraft computational project model.

---

114. VERSION 0.1 ENGINEERING STATE

The OneDraft platform now contains the architectural foundation for:

```
Product Definition
System Architecture
Technology Architecture
Domain Model
Persistence Model
OpenAPI Contract
Project Model
Parametric Geometry
Compliance Engine
Validation Engine
Revision System
Redline System
Documentation Model
Drawing Generation
Sheet Composition
Schedule Generation
Document Provenance
Customer Acceptance
```

The Documentation & Drawing Generation Engine completes the primary path from computational project state to architectural document output.

---

115. NEXT IMPLEMENTATION STAGE

The next engineering stage is executable implementation of the documentation stack.

The first implementation artifacts are:

`documentation_schema.sql`

`documentation_models.py`

`documentation_service.py`

`drawing_generator.py`

`sheet_composer.py`

`schedule_generator.py`

`documentation_validator.py`

`document_package_service.py`

These artifacts establish the first executable documentation subsystem.

---

116. SPECIFICATION STATUS

ONEDRAFT DOCUMENTATION & DRAWING GENERATION ENGINE v0.1

STATUS: ESTABLISHED

The documentation engine is now defined as the controlled translation layer between the computational OneDraft project model and the final architectural drawing package.